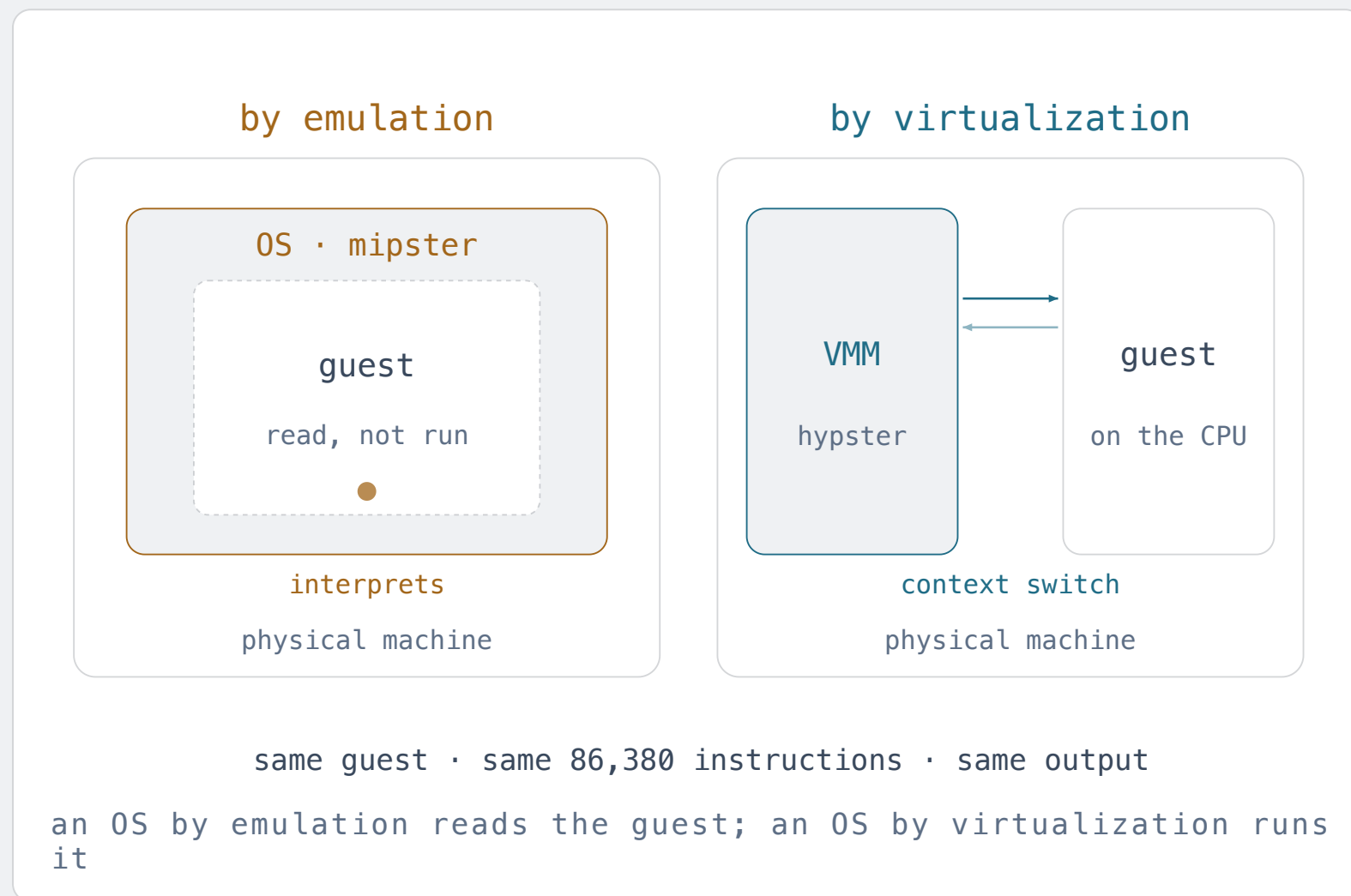


Self-Hosting

Hypster. Emulation equals virtualization. The bootstrap problem is Gödel's second theorem, and the trusted computing base is its axiom.

[← SE 05 · Time-Sharing](#)

Run the guest's instructions on the host's processor. Trap only what crosses the **boundary**.

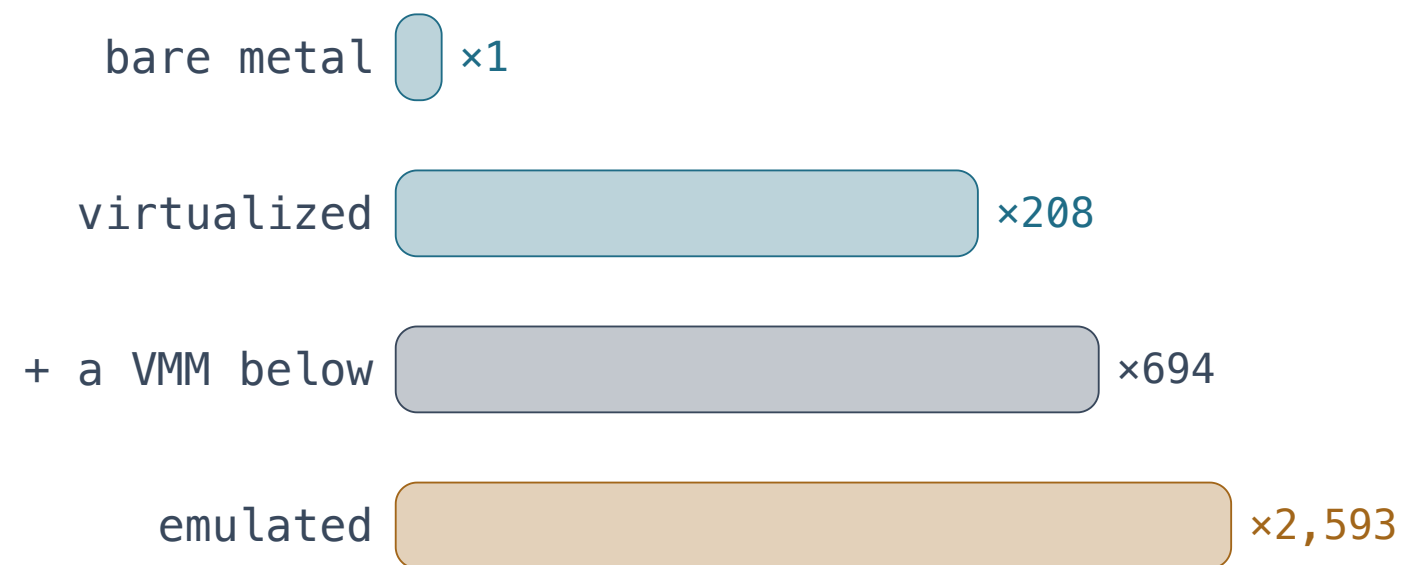


Hypster does what mipster does with one change: it maps the guest's context onto the processor and lets it execute the guest's instructions. Adds, loads, branches: native speed. System calls, page faults, the timer: exceptions back to hypster.

Same signature, same contexts, same handlers. The guest cannot tell which one hosts it: same output, same 86,380 instructions. The emulator is the executable specification.

THE BILL

×2,593 versus ×208.
Performance is the **only** reason.



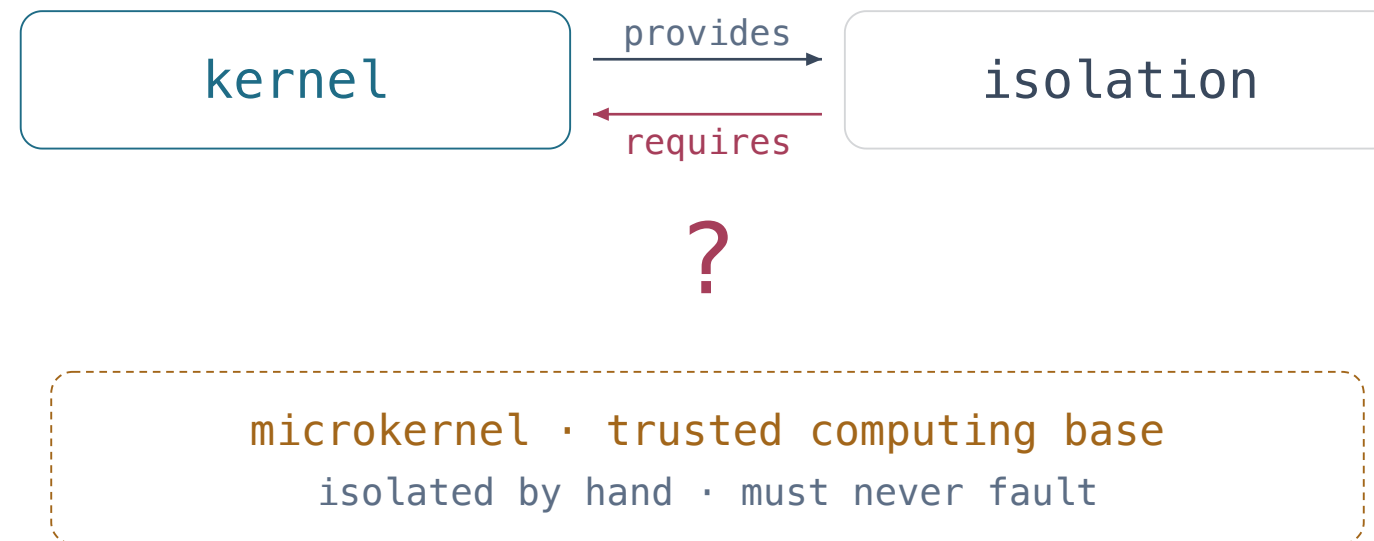
host instructions per guest instruction, emulated and virtualized

```
$ make os-emu  
> selfie.m: ... 86380 executed instructions ...  
./selfie: ... 17860937 executed instructions ...
```

The guest still runs 86,380; the host 18 million instead of 222. What remains is exceptions and context switches: on real hardware, a few percent.

Nothing else changed: not the isolation, not the output, not the semantics. An optimisation preserves meaning. So what does it cost?

The kernel now runs on the machine it isolates. It needs the isolation it **provides**.



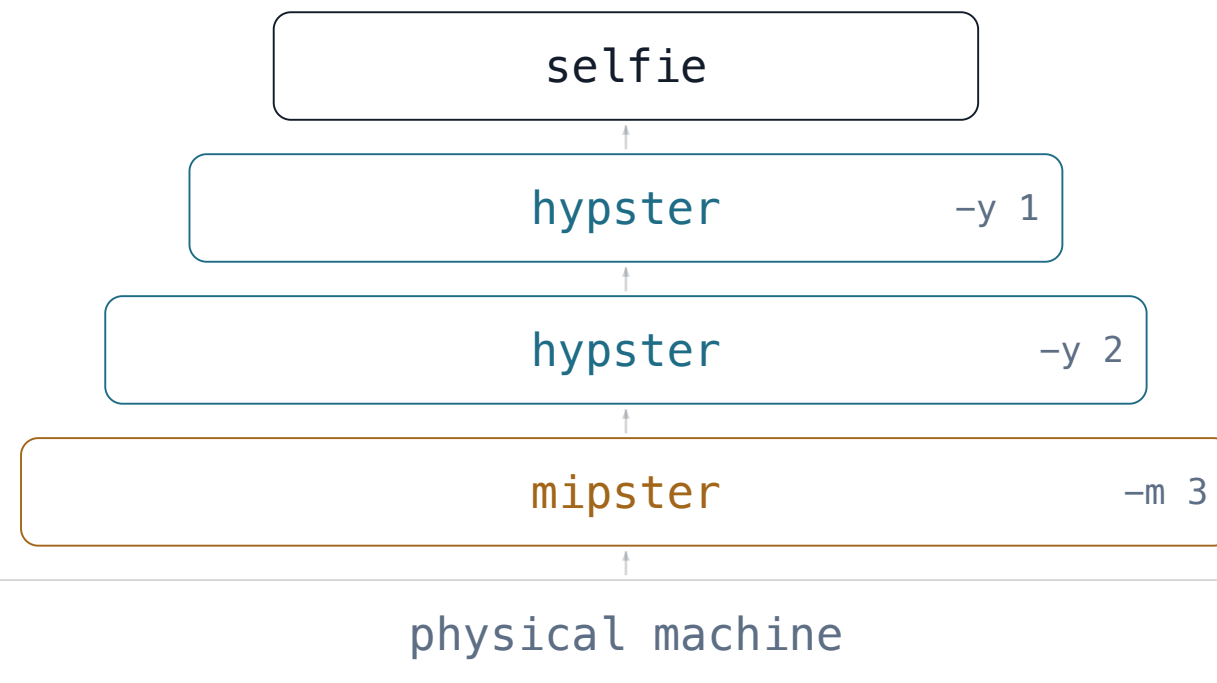
virtualization is emulation plus self-reference

In mipster the guest is data; the host is never at stake. In hypster the guest runs on the host's processor, behind the host's page tables. Who protects the page tables? The page tables.

So the kernel must be isolated from the very things it isolates: memory only it can map, a mode only it can enter, a timer only it can set.

The price is self-reference, where every limit lives.

Hypervisor on hypervisor on emulator. Somebody has to be *first*.



self-hosting: each level virtualizes the one above, and the bottom is emulated

```
$ ./selfie -c selfie.c -o selfie.m -m 3 \
    -l selfie.m -y 2 -l selfie.m -y 1
```

Each level asks the one below to create a context and run it; requests fall to the bottom, where mipster, an emulator, executes.

Gödel's second theorem in a kernel: no system establishes its own consistency from inside, no kernel its own isolation. The ground is outside: hardware privilege, an emulator, a smaller kernel.

What you cannot prove, you **name**. The TCB is the axiom set of a system.

The trusted computing base is everything a system's isolation depends on but cannot itself check: the hardware's privilege mechanism, the bottom of the ladder, the code that switches contexts. Engineering does not eliminate it; it makes it small, and names it.

Selfie's TCB is mipster: a few hundred lines that read the guest. seL4, the verified microkernel, has about ten thousand lines of TCB and a proof of everything above it, a proof that assumes the hardware. Axioms: not proven, chosen.

ASSIGNMENT · FORK-WAIT

Add fork and wait as system calls: a process that copies itself, and a parent that blocks until a child exits. Design this week; week 7 says what a process is and finishes it with exit. `./grader/self.py fork-wait` when it runs.

Listen, then read.

LISTEN

Wagner · Lohengrin — Bayreuth 1982. “Nie sollst du mich befragen”: one thing may not be asked, and the protection lasts exactly as long as nobody asks it. The trusted computing base, and what happens in Act III when Elsa questions it.

READ

The Protection of Information in Computer Systems — Saltzer and Schroeder, 1975: the design principles of protection, and the paper that named the trusted computing base’s problem. And, more technical, seL4: Formal Verification of an OS Kernel — Klein and others, 2009: a kernel proved correct relative to its hardware; read it for what the proof assumes.